

Input / Output & Operators

`cin`, `cout`, all five operator groups, and the `cmath` library

So far your programs have only talked **to** you. Now they will listen as well — and once data is coming in, you need operators to actually do something with it.

Output — `cout`

`cout` displays things on the screen, using the `<<` operator. Think of the arrows as pushing data **towards** the screen.

MAIN.CPP

```
cout << "Hello";  
cout << "Total: " << 50;    // chain as many as you like  
cout << endl;               // move to the next line
```

Input — `cin`

`cin` reads input from the user, using `>>`. The arrows point the other way, because data is flowing **from** the keyboard **into** your variable.

MAIN.CPP

```
int age;  
cout << "Enter age: ";  
cin >> age;  
cout << "Your age is " << age;
```



An easy way to remember the direction: the arrows always point **the way the data is travelling**. Into `cout` and out to the screen; out of `cin` and into your variable.

Operators

Operators are symbols that perform an action on values. C++ groups them like this:

TYPE	OPERATORS	USED FOR
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	Basic math
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code>	Store or update a value
Relational	<code>=</code> <code>≠</code> <code>></code> <code><</code> <code>≥</code> <code>≤</code>	Compare values → true / false
Logical	<code>&&</code> <code> </code> <code>!</code>	Combine conditions
Increment / Decrement	<code>++</code> <code>--</code>	Increase or decrease by 1

Arithmetic Operators

The four you already know behave exactly as expected. Two of them have behaviour worth stopping on.

THE SPECIALITY OF /

Division in C++ depends entirely on the **types** of the two numbers, not on the answer.

INTEGER VS FLOATING DIVISION

```
cout << 7 / 2;           // 3    - both are int, decimal is thrown away
cout << 7.0 / 2;         // 3.5  - one is a double, so result is double
cout << 7 / 2.0;         // 3.5  - same thing
```

⚠ If **both** operands are integers, C++ does integer division and simply drops the decimal part. `7 / 2` is `3`, not `3.5`. To get a decimal answer, at least one side must be a decimal.

THE SPECIALITY OF %

The modulus operator gives the **remainder** after division.

MODULUS

```
cout << 7 % 2;           // 1
cout << 10 % 5;          // 0    - divides evenly
cout << 7.0 % 2;         // ERROR - % does not work on decimals
```

- `%` works **only with integers** in C++. Using it on a `float` or `double` is a compile error.
- Dividing or taking modulus by **zero** crashes the program — always check the divider first.
- `n % 2 == 0` is the standard test for an even number. You saw this exact check in the flowchart notes.

ARITHMETIC BETWEEN CHARACTERS

This one surprises everyone. A `char` does not really store a letter — it stores a **number**, the ASCII code of that letter. So characters can be used in maths.

CHARACTER	ASCII VALUE
'A'	65
'Z'	90
'a'	97
'z'	122
'0'	48

CHAR ARITHMETIC

```
char c = 'A';

cout << c + 1;           // 66 - result is an int
cout << (char)(c + 1);   // B   - cast it back to see the letter

cout << 'a' - 'A';       // 32 - the gap between upper and lower case
```

Two tricks that fall straight out of this, and that you will reuse constantly:

Digit char → number

`'7' - '0'` gives the number `7`,
because $55 - 48 = 7$.

Upper ↔ lower case

`'A' + 32` gives `'a'`. Subtract
32 to go back up.

Assignment Operators

`=` stores a value. The rest are shortcuts that update a variable using its own current value.

OPERATOR	EXAMPLE	SAME AS
<code>=</code>	<code>x = 5</code>	Store 5 in x
<code>+=</code>	<code>x += 3</code>	<code>x = x + 3</code>
<code>-=</code>	<code>x -= 3</code>	<code>x = x - 3</code>
<code>*=</code>	<code>x *= 3</code>	<code>x = x * 3</code>
<code>/=</code>	<code>x /= 3</code>	<code>x = x / 3</code>
<code>%=</code>	<code>x %= 3</code>	<code>x = x % 3</code>

⚠️ `=` and `==` are completely different. `=` **assigns** a value; `==` **compares** two values. Writing `=` where you meant `==` is a classic bug that still compiles.

Relational Operators

These compare two values and always give back `true` or `false`.

OPERATOR	MEANING	EXAMPLE (X = 5)
<code>==</code>	Equal to	<code>x == 5 → true</code>
<code>!=</code>	Not equal to	<code>x != 5 → false</code>

OPERATOR	MEANING	EXAMPLE (X = 5)
>	Greater than	<code>x > 3 → true</code>
<	Less than	<code>x < 3 → false</code>
>=	Greater than or equal	<code>x ≥ 5 → true</code>
<=	Less than or equal	<code>x ≤ 4 → false</code>

Logical Operators

These combine two or more conditions into one answer.

OPERATOR	NAME	GIVES TRUE WHEN
&&	AND	Both conditions are true
	OR	At least one condition is true
!	NOT	Flips the answer — true becomes false

MAIN.CPP

```
int age = 20;

age > 18 && age < 60    // true  - both hold
age < 18 || age > 60    // false - neither holds
!(age > 18)             // false - flips true
```

Increment & Decrement

`++` adds one, `--` subtracts one. But **where** you put it changes what the expression gives back.

Pre — `++x`

Change first, then use the value.

Post — `x++`

Use the value first, then change it.

THE DIFFERENCE

```
int x = 5;
cout << x++;    // prints 5, then x becomes 6

int y = 5;
cout << ++y;    // y becomes 6, then prints 6
```



On its own line, `x++` and `++x` do exactly the same thing — both just add one. The difference only shows up when you **use** the value in the same statement, like inside a `cout`.

The cmath Library

C++ does not have a built-in square root or power operator. Those live in a library called `cmath`, which you include at the top of your file.

MAIN.CPP

```
#include <iostream>
#include <cmath>
using namespace std;
```

Ten functions from it that cover almost everything you will need:

FUNCTION	WHAT IT DOES	EXAMPLE → RESULT
sqrt(x)	Square root	sqrt(25) → 5
cbrt(x)	Cube root	cbrt(27) → 3
pow(x, y)	x raised to the power y	pow(2, 3) → 8
fabs(x)	Absolute value (removes the minus)	fabs(-4.5) → 4.5
ceil(x)	Rounds up to the next whole number	ceil(4.1) → 5
floor(x)	Rounds down	floor(4.9) → 4
round(x)	Rounds to the nearest whole number	round(4.5) → 5
trunc(x)	Chops the decimal off	trunc(4.9) → 4
fmod(x, y)	Remainder for decimals	fmod(7.5, 2) → 1.5
log10(x)	Logarithm to base 10	log10(1000) → 3



`fmod` is the answer to the earlier problem — `%` refuses to work on decimals, so when you need a remainder from a `double`, this is the function to reach for.

MAIN.CPP

```
double n = 25;
cout << sqrt(n);           // 5
cout << pow(2, 10);        // 1024
cout << ceil(4.1);         // 5
cout << floor(4.9);        // 4
```